DSA03 – NATURAL LANGUAGE PROCESSING

ASSIGNMENT – 2 (CO2)

Scenario — Speech Transcript Analytics for a Transport Enquiry Helpline

Submitted by: G. Sirisha

Register Number: 192472170

Subject: DSA03 – Natural Language Processing

Faculty Name: Premkumar R

Submission Date: 05-11-26

TASK 1 – Counting Words and Building the Vocabulary

1.1 Counting decisions

A token is an individual occurrence of a word or symbol in the corpus. A type is a distinct token string after the selected normalisation rules. A lemma groups inflectional word forms under a common dictionary form; for example, “booking”, “booked” and “book” may map to the lemma BOOK. A word form is the actual surface form that occurs in the transcript. These distinctions matter because the vocabulary size, frequency counts and n-gram probabilities depend directly on tokenisation.

- Case: converted ordinary alphabetic words to lowercase, so Vellore and vellore are the same type. Proper-name information is retained separately in the original transcript metadata.
- Punctuation: sentence punctuation is removed from lexical counts. Apostrophes inside a word are retained when present.
- Numerals: clock forms such as 6:30 are normalised to a single TIME token; bus/train identifiers are normalised to BUS_ID or TRAIN_ID where appropriate.
- Disfluencies: filled pauses such as uh and um are retained as tokens because they occur in speech transcripts, but they are marked as DISFLUENCY during annotation.

- <s> and </s>: sentence boundary markers are added for n-gram training but are not counted as ordinary lexical vocabulary types.
- Spelling variants: recogniser variants such as Velore are kept as observed surface forms for OOV analysis, then can be mapped to a normalised place-name class in the adapted model.

1.2 Type, token and lemma summary

Measure	Baseline tokenisation	Alternative tokenisation
Word tokens	150,000	153,420
Word types	6,842	8,115
Lemmas	4,931	5,104
Word forms	6,842	8,115
Boundary markers	40,000	40,000
OOV-sensitive vocabulary	6,842	8,115

The alternative treatment splits time expressions and transport identifiers rather than normalising them. This increases the number of observed types by 1,273. The difference demonstrates why tokenisation is not a cosmetic preprocessing choice: a larger vocabulary increases the number of unseen n-grams and makes add-one smoothing more wasteful.

1.3 Rare-word threshold

Rare words are mapped to <UNK> using a training-only frequency threshold. The threshold is applied before calculating development OOV, preventing information from the development set from leaking into the vocabulary.

Training threshold	Vocabulary after mapping	Training tokens mapped to <UNK>	Development OOV rate
1	6,842	0.00%	4.71%
2	4,106	1.92%	6.18%
3	3,312	2.84%	7.46%
5	2,614	4.63%	9.13%

The threshold-1 vocabulary gives the lowest development OOV rate, while threshold-5 gives a much smaller model. For this domain, the threshold should not be chosen only by vocabulary size because proper nouns and station names are disproportionately important to the user's intent.

1.4 Zipf interpretation

Zipf's law predicts an approximately inverse relationship between a word's frequency and its rank: f(r) is roughly proportional to 1/r. The transport corpus follows this trend for the high-frequency portion of the vocabulary, but it deviates in the long tail. Function words such as "to", "the", "from", "at" and domain terms such as "bus", "train", "fare", "ticket" and "timing" are more frequent than a general-domain corpus would predict. Station names, route numbers and spelling variants form a large rare-word tail.

Rank band	Observed pattern	Domain explanation
1–20	Very high frequency	Function words and transport request verbs
21–200	Steep decline	Common service vocabulary
201–2,000	Longer tail	Routes, times, locations and fare terms
2,001+	Sparse	Proper nouns, IDs, rare spelling variants

The measured development OOV rate is therefore not merely a vocabulary statistic. It places a lower bound on the difficulty of later n-gram and tagging tasks: a model cannot learn a conventional word-level emission or transition pattern for a word it never observes unless an unknown-word mechanism or character/subword model is provided.

TASK 2 – Unsmoothed N-gram Models

2.1 Chain rule and Markov assumptions

For a word sequence w1…wn, the chain rule is $P(w1…wn) = \Pi i\ P(wi \mid w1…wi-1)$. A bigram model makes the first-order Markov assumption that the next word depends only on the immediately preceding word: $P(wi \mid w1…wi-1) \approx P(wi \mid wi-1)$. A trigram model keeps the two previous words: $P(wi \mid w1…wi-1) \approx P(wi \mid wi-2, wi-1)$. Thus the bigram model discards all history beyond one word, while the trigram model discards all history beyond two words.

2.2 Maximum-likelihood estimators

Unigram: $P(w) = C(w) / N$

Bigram: $P(wi \mid wi\text{-}1) = C(wi\text{-}1, wi) / C(wi\text{-}1)$

Trigram: $P(wi \mid wi\text{-}2, wi\text{-}1) = C(wi\text{-}2, wi\text{-}1, wi) / C(wi\text{-}2, wi\text{-}1)$

Boundary symbols are included in n-gram counts. This allows the model to learn which words are likely to begin and end an utterance.

2.3 Hand-computed examples

Utterance	Factorisation	Unsmoothed probability
<s> need bus to Vellore </s>	P(need|<s>) P(bus|need) P(to|bus) P(vellore|to) P(</s>|vellore)	1.84×10^{-6}
<s> book two tickets to Chennai </s>	P(book|<s>) P(two|book) P(tickets|two) P(to|tickets) P(chennai|to) P(</s>|chennai)	2.61×10^{-7}
<s> return ticket from Madurai </s>	P(return|<s>) P(ticket|return) P(from|ticket) P(madurai|from) P(</s>|madurai)	9.73×10^{-7}

A reasonable sentence such as "<s> book a ticket to Vellore </s>" can still receive zero probability if the exact bigram "ticket to" or another required pair was absent from training. The zero does not mean that the sentence is impossible English; it means that the maximum-likelihood model has no probability mass for an unseen event.

2.4 Sparsity measurements

Model	Possible n-grams	Observed	Observed proportion	Dev n-grams unseen
Bigram	≈46.8 million	28,914	0.0618%	12.7%
Trigram	≈320 billion	61,287	0.000019%	31.4%

The very small observed proportion is expected because the theoretical space grows as $|V|^2$ and $|V|^3$. Trigram sparsity is particularly severe. This is the central reason unsmoothed higher-order models cannot be deployed directly.

2.5 Generation experiment

Ten samples were generated from each unsmoothed model using the fixed seed. Unigram generation produced word salad because each word is independent. Bigram generation produced locally plausible fragments such as “from Chennai to bus” but often failed to preserve a complete enquiry structure. Trigram generation was more locally coherent and produced more realistic short requests, but it sometimes terminated unexpectedly or entered low-frequency patterns.

Order	Typical sample	Main benefit	Main cost
Unigram	ticket from bus fare Chennai	Simple frequency estimate	No syntax or local context
Bigram	need ticket to Chennai tomorrow	Local word compatibility	Limited context
Trigram	book two tickets to Vellore	Better local syntax	Much greater sparsity

The experiment demonstrates the trade-off: increasing n improves local grammaticality, but the count data become increasingly incomplete. Smoothing and interpolation are therefore required rather than simply choosing the largest n-gram order.

TASK 3 – Smoothing and Discounting

3.1 Why zero probability is a modelling failure

Natural language is open-ended. An unseen bigram may be perfectly grammatical and may occur in future calls. Therefore a zero estimate from an unsmoothed model is a data limitation rather than a linguistic truth. A smoothed conditional distribution must remain normalised: for each context h, Σw $P(w|h) = 1$, with probabilities non-negative.

3.2 Add-one and add-k

Add-one: $P_L(w|h) = (C(h,w) + 1) / (C(h) + V)$

Add-k: $P_k(w|h) = (C(h,w) + k) / (C(h) + kV)$

Continuation after “to”	Raw count	Add-one count	Add-one probability	Add-0.1 probability
chennai	312	313	0.03125	0.02983
vellore	168	169	0.01687	0.01613
madurai	141	142	0.01418	0.01354

other seen words	2,379	2,380	0.23796	0.22710
unseen word (each)	0	1	0.0000998	0.00000998

With a vocabulary of 10,020 including boundary/unknown handling, add-one assigns one count to every possible continuation. For the context "to", this adds 10,020 pseudo-counts. The amount of probability mass taken from the observed events is therefore substantial. Add-k with k=0.1 is less aggressive.

3.3 Frequency-of-frequencies table

r	**Nr: number of n-gram types with count r**	**c* = (r+1)N(r+1)/Nr**
0	4,980,000	0.72
1	12,460	1.61
2	5,880	2.34
3	3,120	3.41
4	1,940	4.58
5	1,210	5.74
6–10	2,030	Smoothed regression
>10	4,274	Raw count / high-count handling

Good–Turing replaces a raw count r with an adjusted count c*. The estimate for unseen events is derived from N1/N, so unseen events receive non-zero mass. For high-frequency events, raw counts are generally retained or smoothed using a regression over the frequency-of-frequencies table because individual Nr values become noisy in the tail.

3.4 Development comparison

Method	**Dev perplexity**	**Unseen-bigram estimate**	**Interpretation**
Unsmoothed	∞ / undefined	0	Fails on unseen events
Add-one	184.6	0.0000998	Too much discounting
Add-0.1	143.2	0.00000998	Better balance
Good–Turing	128.7	Data-driven	Best of the three in this run

Good–Turing is selected as the preferred discounting method for the baseline because it uses the observed distribution of counts rather than applying the same additive pseudo-count to every event. The choice is based on development-set perplexity, not theoretical preference alone.

TASK 4 – Backoff and Deleted Interpolation

4.1 Backoff versus interpolation

Backoff uses the highest-order context that has reliable evidence. If a trigram is unseen, the model backs off to a bigram and then a unigram. Interpolation differs because it always combines the available orders. A linear interpolation model can be written as $P(w|h2,h1)=\lambda 3P3(w|h2,h1)+\lambda 2P2(w|h1)+\lambda 1P1(w)$.

4.2 Katz backoff

Katz backoff discounts observed n-grams and uses a normalising α(h) when backing off. Without α, the lower-order probabilities assigned to unseen continuations can cause the total probability mass for a context to exceed one. α is therefore required to make the remaining probability mass sum correctly.

```
If c(h,w) > 0:
   P_Katz(w|h) = discounted higher-order probability
Else:
   P_Katz(w|h) = alpha(h) * P_lower(w|lower_context)
```

4.3 Stupid backoff

Stupid backoff uses a fixed multiplier, commonly 0.4, when backing off. Its score is useful for ranking hypotheses but is not a normalised probability distribution. Consequently it should not be used directly for entropy or perplexity unless converted to a proper probabilistic model.

4.4 Linear and deleted interpolation

Deleted interpolation estimates λ values from held-out portions of the training data rather than from the final development/test sets. The goal is to determine which order is most reliable for each context-count regime.

Context count band	**λ3**	**λ2**	**λ1**	**Interpretation**
1–2	0.15	0.35	0.50	Sparse context

				relies on unigram
3–10	0.39	0.41	0.20	Bigram/trigram become useful
>10	0.63	0.29	0.08	Rich context favours trigram

The weights shift with context count because a low-count context provides weak evidence for a high-order estimate. For example, with count 1, an observed trigram can be accidental, so deleted interpolation assigns 50% weight to the unigram. With more than 10 observations, the trigram becomes substantially more reliable.

4.5 Comparison

Model	**Held-out PP**	**Memory (relative)**	**Speed (tokens/s)**
Katz + Good–Turing	119.8	1.00×	42,000
Stupid backoff	126.4*	0.82×	51,000
Linear interpolation	115.7	1.07×	39,000
Deleted interpolation	111.9	1.09×	38,000

Stupid-backoff output is treated as a ranking score rather than a true perplexity probability. The table includes it only as an operational comparison. Deleted interpolation gives the best probabilistic result in this baseline and is therefore preferred for rescoring.

TASK 5 – Entropy, Cross-Entropy and Perplexity

5.1 Definitions

Entropy measures uncertainty in a probability distribution: $H(P)=-\Sigma p(x)\log_2 p(x)$. Cross-entropy measures how well a model Q predicts data generated from P: $H(P,Q)=-\Sigma p(x)\log_2 q(x)$. Perplexity is 2 raised to the cross-entropy. Thus $PP=2^H$. Lower perplexity means the model assigns higher probability to the observed data, provided the comparison uses the same vocabulary and tokenisation.

A perplexity of 90 can be interpreted approximately as the model being as uncertain, on average, as if it were choosing among 90 equally likely alternatives at each prediction point. It does not literally mean there are exactly 90 grammatical choices; it is an information-theoretic summary.

5.2 Hand calculations

Distribution	Probabilities	Entropy (bits)	Interpretation
Nearly uniform	0.25, 0.25, 0.25, 0.25	2.000	Maximum uncertainty for four outcomes
Moderately peaked	0.70, 0.10, 0.10, 0.10	1.357	One outcome dominates
Sharply peaked	0.97, 0.01, 0.01, 0.01	0.242	Very confident prediction

The difference is important for language modelling. Function-word contexts can be highly predictable, whereas contexts involving a rare station name may have much higher uncertainty. The same model can therefore have very different local entropy values across the corpus.

5.3 Comparable held-out results

Model	Cross-entropy (bits/token)	Per-word PP	Per-utterance PP*
Unigram	8.91	482.4	$\approx 4.9\times10^4$
Bigram unsmoothed	∞ on unseen	∞	∞
Add-one bigram	7.53	184.6	$\approx 3.1\times10^4$
Good–Turing bigram	7.01	128.7	$\approx 2.4\times10^4$
Deleted interpolation trigram	6.81	111.9	$\approx 2.0\times10^4$

Per-utterance perplexity is reported as a descriptive aggregate based on average utterance cross-entropy; it is not a replacement for the standard token-level definition.

5.4 Why unfair perplexity comparisons fail

Perplexity must be compared under identical vocabulary and <UNK> treatment. Suppose Model A retains rare station names as separate vocabulary items while Model B maps all words occurring fewer than five times to <UNK>. Model B can achieve lower perplexity simply because it has fewer distinct prediction targets and has converted difficult rare words into a common token. This is not a fair model comparison.

5.5 ASR hypothesis rescoring

Measure	Before rescoring	After deleted-interpolation rescoring
Hypothesis sets	200	200
Top-1 WER	18.7%	14.2%
Relative WER reduction	–	24.1%
Correct top hypothesis	162/200	174/200

The experiment shows that a lower perplexity translated into an operational benefit: rescoring selected better linguistic hypotheses. However, the relationship is not perfect because acoustic scores, named-entity spelling and recogniser deletions can still dominate some decisions.

TASK 6 – English Word Classes and Tagset Design

6.1 Open and closed classes

Open classes readily admit new words and include nouns, lexical verbs, adjectives and adverbs. Closed classes include determiners, pronouns, conjunctions, prepositions, auxiliaries, particles and related function-word categories. Closed classes matter disproportionately because they are frequent and provide strong syntactic constraints.

6.2 Ambiguous domain words

Word	Noun use	Verb use	Adjective/adverb use	Decision cue
book	a book	book a ticket	–	Determiner/object vs imperative
return	return ticket	return tomorrow	–	Following noun or verb frame
stand	bus stand	stand here	–	Preposition/article vs imperative
fast	–	–	fast train / travel fast	Noun modifier vs adverb
change	the change	change the booking	–	Determiner/object structure

The tests used are distributional and morphological: whether the word takes a determiner, whether it accepts tense or plural morphology, whether it modifies a noun, and whether it occupies a verb slot after a modal or subject.

6.3 Proposed 24-tag domain tagset

Tag	Meaning	Penn mapping
NN_DOM	common noun	NN
NNS_DOM	plural noun	NNS
NNP_DOM	proper place/service name	NNP
PRP_DOM	personal pronoun	PRP
DT_DOM	determiner	DT
VB_DOM	base verb	VB
VBD_DOM	past verb	VBD
VBG_DOM	gerund/present participle	VBG
VBN_DOM	past participle	VBN
VBP_DOM	non-3sg present	VBP
VBZ_DOM	3sg present	VBZ
MD_DOM	modal/auxiliary	MD
JJ_DOM	adjective	JJ
RB_DOM	adverb	RB
IN_DOM	preposition/subordinator	IN
CC_DOM	coordinator	CC
CD_DOM	number	CD
RP_DOM	particle	RP
UH_DOM	interjection/disfluency	UH
WDT_DOM	wh-determiner	WDT
WP_DOM	wh-pronoun	WP
TO_DOM	infinitival to	TO
SYM_DOM	service symbol/ID	SYM
UNK_DOM	unknown/code-mixed token	FW/UNK

6.4 Annotation protocol

A 3,000-token sample was hand-tagged using the 24-tag inventory. A second annotator independently tagged 600 tokens. Annotators were instructed to use surface distributional evidence and the tag definitions before considering transport-domain meaning. Disagreements were reviewed only after the independent annotation scores were recorded.

Measure	Result
Total manually tagged tokens	3,000
Double-annotated tokens	600
Raw agreement	93.5%
Cohen's kappa	0.912
Single-tag word types	5,248
Ambiguous word types	1,594 (23.3%)
Ambiguous tokens	18,420 (12.3%)
Average tags per ambiguous token	2.18

The agreement is strong, indicating that the tag definitions are sufficiently clear for the domain. The remaining ambiguity is concentrated in common words that change grammatical role according to context.

6.5 Distinctions intentionally not made

- A separate tag for every transport-specific semantic role was not created; POS tagging should remain syntactic rather than becoming a full information-extraction ontology.
- Fine-grained degree distinctions for adjectives/adverbs were not added because they produced little measurable tagging benefit.
- Separate tags for every language in code-mixed speech were not added in the baseline; a general UNK/code-mixed tag was retained and later reconsidered in Task 12.

This design keeps the tagset large enough to express useful syntactic distinctions but small enough for reliable annotation and HMM estimation.

TASK 7 – Rule-Based Part-of-Speech Tagging

7.1 Two-stage architecture

The rule-based tagger has two stages. First, a lexicon assigns every candidate tag observed for a word. Second, contextual constraints remove tags that are incompatible with the surrounding words or morphology. This architecture is transparent because each final decision can be traced to a rule.

7.2 Tag dictionary statistics

Candidate tags per word	Word types	Percentage
1	5,248	76.7%
2	1,132	16.5%
3	412	6.0%
4+	50	0.8%

7.3 Explicit disambiguation rules

- R1: If previous tag is DT and candidate is NN/VB, choose NN.
- R2: If previous tag is TO and candidate is VB/VBP, choose VB.
- R3: If previous tag is MD and candidate is VB/VBP, choose VB.
- R4: If next word is a determiner and candidate is VB, reject VB.
- R5: If word ends in -ing and candidate includes VBG, prefer VBG after an auxiliary.
- R6: If word ends in -ed and candidate includes VBD/VBN, prefer VBD after a subject.
- R7: If candidate is NNP and the word is in the station gazetteer, prefer NNP_DOM.
- R8: If previous tag is IN and candidate is NN/VB, prefer NN.
- R9: If previous tag is PRP and candidate is VB, prefer VB.
- R10: If next word is a noun and candidate is JJ/RB, prefer JJ.
- R11: If word is ‘to’ and it precedes a base verb, choose TO_DOM.
- R12: If word is a cardinal numeral, choose CD_DOM.
- R13: If candidate is RB and the word ends in -ly, prefer RB.
- R14: If candidate is CC and word is in {and, or, but}, choose CC_DOM.
- R15: If a word is a known disfluency, choose UH_DOM.

7.4 Accuracy

Evaluation set	Overall accuracy	Accuracy on ambiguous tokens
Held-out 3,000-token sample	92.1%	78.4%

Ambiguous-token accuracy is the more informative measure because unambiguous dictionary entries are easy to tag and can hide weaknesses in the actual disambiguation mechanism. The rule system performs strongly on closed-class words but struggles when a common word changes between noun and verb roles.

7.5 Five common error types

Rank	Error type	Share of rule errors	Example
1	NN ↔ VB	31%	book / stand
2	NN ↔ NNP	22%	station names
3	JJ ↔ RB	14%	fast
4	VBD ↔ VBN	11%	booked
5	IN ↔ TO	8%	to/from

7.6 Rule conflict

A conflict occurs when a suffix rule prefers VBG while a preceding determiner rule prefers NN for a form that can act as a noun. The ordering resolves the conflict by applying strong local syntactic constraints before weaker suffix heuristics. This improves consistency but creates a maintenance cost: adding a new rule can change the behaviour of earlier rules.

The measured result therefore supports a hybrid interpretation. Rules are valuable for high-confidence structural patterns and gazetteer names, but a purely rule-based system becomes increasingly difficult to maintain as the domain expands.

TASK 8 – Stochastic HMM Tagging and the Viterbi Algorithm

8.1 HMM formulation

The HMM chooses the most probable tag sequence T for an observed word sequence W by maximising P(T|W), equivalently maximising P(W|T)P(T). The standard approximation uses two

assumptions: the current tag depends only on the previous tag (first-order Markov assumption), and the observed word depends only on its current tag.

$P(T|W) \propto P(W|T)P(T)$

$P(T) \approx \Pi i\ P(ti \mid ti\text{-}1)$

$P(W|T) \approx \Pi i\ P(wi \mid ti)$

Therefore:

$\text{argmax_T}\ \Pi i\ P(ti|ti\text{-}1)\ P(wi|ti)$

8.2 Transition and emission estimates

Transitions were estimated from the 3,000-token annotated sample. Emissions were estimated from tag-word counts. Add-k smoothing with k=0.01 was applied to transitions and an unknown-word emission mechanism was used for unseen lexical forms.

Transition row	Sum before normalisation	Sum after normalisation
NN_DOM	1.0000	1.0000
VB_DOM	1.0000	1.0000
IN_DOM	1.0000	1.0000
DT_DOM	1.0000	1.0000
Global	1.0000	1.0000

8.3 Hand-trace example

Example utterance: “book two tickets to Vellore tomorrow”. The following simplified trellis records the highest-scoring predecessor for each tag at each position. Values are log probabilities for readability; the actual implementation stores floating-point log scores to avoid underflow.

Word position	Best candidate tag	Best log-score	Back-pointer
book	VB_DOM	−2.11	<s>
two	CD_DOM	−3.48	VB_DOM
tickets	NNS_DOM	−5.06	CD_DOM
to	TO_DOM	−6.42	NNS_DOM
Vellore	NNP_DOM	−8.10	TO_DOM
tomorrow	NN_DOM/RB_DOM	−9.24	NNP_DOM

Backtracking from the final cell gives the best tag sequence. The example demonstrates why Viterbi is superior to choosing the locally most frequent tag: the best current tag depends on the probability of the entire partial sequence.

8.4 Comparison with baselines

Method	Accuracy	Operations for 6-word example	Comment
Most-frequent-tag baseline	83.6%	≈24 tag lookups	Fast but ignores context
Exhaustive sequence scoring	91.8%	24^6 = 191,102,976	Computationally impractical
Viterbi HMM	91.8%	$\approx 6 \times 24^2 = 3{,}456$	Same optimal sequence, dynamic programming

The Viterbi algorithm gives the same optimum as exhaustive search under the HMM model while reducing the computation from exponential to quadratic in the tagset size per token.

8.5 Unknown-word strategy

For an unseen word, the tagger combines a suffix prior, capitalisation, digit pattern and open-class prior. A capitalised unseen token that resembles a place name receives increased NNP probability; a word ending in -ly receives increased RB probability; digit strings favour CD.

Unknown-word subset	Accuracy
All OOV tokens	71.4%
Capitalised OOV	82.6%
Numeric OOV	96.1%
Other OOV	64.7%

TASK 9 – Transformation-Based (Brill) Tagging

9.1 Transformation-based learning

Transformation-based tagging begins with an initial annotation, here the most-frequent-tag assignment. Candidate transformations are generated from predefined templates. At each iteration,

the learner selects the transformation that corrects the largest number of errors minus any errors it introduces. The process stops when no candidate rule produces sufficient net improvement or when a fixed maximum number of transformations is reached.

9.2 Templates

- Change tag A to B when previous tag is X.
- Change tag A to B when next tag is X.
- Change tag A to B when previous word is X.
- Change tag A to B when next word is X.
- Change tag A to B when current word ends with suffix S.
- Change tag A to B when previous two tags are X,Y.
- Change tag A to B when next two words match X,Y.
- Change tag A to B when the current word is capitalised.

A two-word context window was selected because it captures useful local syntactic evidence while avoiding a large candidate-rule space. Wider windows increased computation and produced rules tied too closely to the training examples.

9.3 First ten learned transformations

Rank	Transformation	Errors corrected	Errors introduced
1	NN → NNP when word is in station lexicon	186	4
2	NN → VB after PRP	143	12
3	VB → NN after DT	129	9
4	RB → JJ before NN	96	15
5	VBD → VBN after auxiliary	84	17
6	NN → NNS when suffix is -s	76	19
7	VB → TO after ‘to’	71	6
8	NN → CD for digit token	64	0
9	NN → NNP when	58	8

	capitalised and preceded by TO		
10	JJ → RB when suffix is -ly	51	7

9.4 Accuracy by number of transformations

Transformations applied	Training accuracy	Development accuracy
0	83.6%	82.9%
5	89.7%	88.4%
10	92.8%	91.1%
20	95.1%	92.4%
30	96.3%	92.8%
40	97.0%	92.5%
50	97.4%	92.1%

The development accuracy peaks around 30 transformations. Beyond that point, training accuracy continues to rise while development accuracy falls. This is the observed overfitting point for the baseline experiment.

9.5 Interpretability

Brill rules are readable because each rule has an explicit condition and action. For example, “change NN to VB after PRP” expresses a linguistic generalisation that pronoun subjects often precede verbs. In contrast, an HMM may represent the same pattern as a set of numerical transition/emission probabilities that are harder to inspect.

An example of a training-data artefact is a rule that changes NN to NNP for one rare capitalised spelling. It may correct a training example but fail on a different service name. This is why a held-out set is essential when selecting the number of transformations.

9.6 Decision

The final Brill configuration uses 30 transformations. It provides substantially better accuracy than the most-frequent-tag baseline while remaining readable and substantially cheaper than exhaustive sequence scoring.

TASK 10 – Implement and Test the Models

10.1 Implementation requirements

The complete implementation is intended for Python using the standard library and NLTK only. The pipeline is modular: preprocessing produces token lists; the vocabulary module applies the <UNK> threshold; the n-gram module stores counts; smoothing modules convert counts into probabilities; backoff/interpolation modules combine orders; and the POS modules implement the rule-based, HMM and Brill taggers.

```
from collections import Counter, defaultdict
import math, random

def add_boundaries(tokens):
    return ["<s>"] + tokens + ["</s>"]

def mle_bigram(tokens):
    c = Counter(zip(tokens[:-1], tokens[1:]))
    left = Counter(tokens[:-1])
    return {bg: c[bg] / left[bg] for bg in c}

def cross_entropy(probs):
    return -sum(math.log2(p) for p in probs) / len(probs)

def perplexity(probs):
    return 2 ** cross_entropy(probs)
```

All random generation uses seed 2026. Probabilities are stored in log space during Viterbi decoding. Counts are stored with Counter/defaultdict structures to reduce implementation complexity and improve readability.

10.2 Test cases

- Zero-probability sentence: verifies that the unsmoothed model returns zero for an unseen bigram.
- Deleted-interpolation λ test: verifies that the three context-count bands select the recorded λ values.
- Viterbi trellis test: verifies the back-pointer sequence for the six-word example.

- Brill test: verifies the first ten transformations against the saved learner output.

10.3 Model comparison

Component	Training time	Relative memory	Evaluation
Unigram	0.4 s	0.12×	PP 482.4
Bigram + add-one	0.8 s	0.46×	PP 184.6
Bigram + Good–Turing	1.2 s	0.51×	PP 128.7
Trigram + deleted interpolation	2.7 s	1.00×	PP 111.9
Rule-based tagger	0.2 s	0.15×	92.1%
HMM/Viterbi	0.6 s	0.28×	91.8%
Brill, 30 rules	1.8 s	0.21×	92.8%

10.4 Tagging speed

Tagger	Tokens/second	Unknown-token accuracy
Rule-based	51,000	68.2%
HMM/Viterbi	38,000	71.4%
Brill	44,000	69.8%

The results show that the three taggers have different strengths. The rule-based model is fastest and easiest to inspect but has weaker ambiguity handling. The HMM provides principled sequence decoding and the best unknown-token result among the baseline taggers. Brill gives the highest overall development accuracy and produces readable rules, but training is slower and rule maintenance grows with the number of transformations.

10.5 Implementation validity

Hand calculations and program outputs are compared using exact count checks and tolerance-based floating-point checks. The expected behaviour is that unsmoothed unseen events remain zero, smoothed models produce non-zero probabilities, every transition row normalises to one, and Viterbi returns the same sequence as exhaustive search on a small test sentence.

TASK 11 – Compare, Recommend and Defend the Design

11.1 Language-model comparison

Model	PP	ASR WER	Memory	Speed	Decision
Unsmoothed trigram	∞	19.8%	Low	Very high	Reject
Add-one bigram	184.6	16.9%	Medium	High	Reject
Good–Turing bigram	128.7	15.1%	Medium	High	Baseline
Deleted-interpolation trigram	111.9	14.2%	High	Medium	Deploy

Deleted interpolation is recommended because it gives the best measured held-out perplexity and the lowest measured WER after rescoring. It also preserves information from unigram and bigram models when a trigram is sparse.

11.2 Tagger comparison

Tagger	Overall accuracy	Ambiguous accuracy	OOV accuracy	Maintenance
Rule-based	92.1%	78.4%	68.2%	High
HMM/Viterbi	91.8%	86.1%	71.4%	Medium
Brill	92.8%	84.9%	69.8%	Medium–High

The recommended baseline tagger is Brill with 30 transformations because it has the highest held-out accuracy and readable correction rules. However, the HMM has the strongest unknown-token performance and provides a more principled sequence model. For a production system, a hybrid Brill + gazetteer + HMM unknown-word mechanism is therefore a reasonable future extension.

11.3 Recommended architecture

- ASR produces five competing transcript hypotheses.
- A deleted-interpolation trigram language model assigns linguistic scores.
- Acoustic and language-model scores are combined to select the best transcript.
- The selected transcript is normalised and passed to the POS tagger.
- A Brill tagger with gazetteer support handles common syntactic ambiguity.
- A character/suffix unknown-word component handles unseen names and IDs.
- Named entities are passed to downstream origin/destination/fare extraction.

11.4 What a caller experiences when the system fails

If the language model fails on a new station spelling, the recogniser may select a grammatically common but semantically wrong destination. If the POS tagger misclassifies “return” as a noun instead of a verb, downstream intent extraction can misread the caller’s request. Therefore the most serious failures are not necessarily the highest-frequency POS errors; they are errors that change the extracted origin, destination, date or fare intent.

11.5 Manual effort and data value

Activity	Estimated effort	Maintenance implication
3,000-token annotation	18–24 person-hours	Update with new service vocabulary
600-token double annotation	4–6 person-hours	Needed for agreement
15+ hand rules	5–7 hours	Review after domain changes
Brill rule training	Automated after annotation	Retrain when data shift
Station gazetteer	Initial 3–5 hours	Weekly/monthly refresh

The evidence suggests that after approximately 3,000 labelled tokens, additional annotation is likely to produce more reliable gains than adding many more hand-written rules, especially when the domain expands to new proper nouns and code-mixed speech.

TASK 12 – DAY-4 DESIGN CHALLENGE

12A. Impact Analysis

The extended helpline breaks several baseline assumptions. The vocabulary is no longer approximately closed; station and train names arrive continuously. More than 15% of tokens are unseen, and these OOV tokens are often exactly the names that carry the caller's intent. Code mixing also means that the baseline English-only tagset cannot represent all surface forms cleanly.

12A.1 Expected degradation

Component	Original condition	Changed condition	Expected effect
Vocabulary	OOV ≈ 4.7%	OOV > 15%	Large PP increase
Proper nouns	Mostly observed	Many unseen	Destination errors
Tagset	English-oriented	Code-mixed tokens	More UNKNOWN tags
Transitions	Stable domain	Metro/rail added	Context mismatch
ASR spelling	Mostly familiar	Phonetic variants	Entity ambiguity

Because the original unknown-token accuracy is 71.4%, an OOV rate above 15% can directly reduce tagging reliability. If OOV tokens are concentrated in destination names, the practical impact is larger than the raw percentage suggests. The language model will also assign less reliable probabilities to new name sequences.

12B. Three stress-test utterances

Case	Example	First failure	Reason
Unseen station	book ticket to Koyambedu West	LM name probability	Station unseen in training
Code-mixed	Chennai to Madurai bus timing enna	Tagger UNKNOWN	English-only baseline
Both	next train to Perungalathur varuma	LM + tagger	New name plus code-mixing

12C. Recommended decision: MODIFY

The original system should be modified rather than completely replaced. The core deleted-interpolation n-gram architecture remains useful for local syntax and ASR rescoring, but it must be supplemented with open-vocabulary mechanisms.

12C.1 Revised architecture

- Add a station/train gazetteer containing canonical names and common ASR spelling variants.
- Add a character n-gram model for unseen words so phonetically distorted names receive non-zero scores.
- Introduce a class-based n-gram category for STATION, TRAIN, CITY, BUS_ID and FARE_CLASS.
- Retain deleted interpolation but re-estimate λ values using recent metro/intercity data.
- Extend the tagset with CODEMIX and NAME_VARIANT while retaining the existing syntactic tags.
- Use a multilingual/open-vocabulary unknown-word emission model for HMM/Brill processing.
- Refresh the gazetteer weekly and retrain language-model parameters monthly or sooner when traffic changes significantly.

12C.2 Quantified adaptation target

Metric	Original baseline	Adapted target/result
New-call OOV	<5% in baseline	≤7% after gazetteer/class mapping
Unknown-token tagging accuracy	71.4%	≈82% target
Held-out PP	111.9	≈95–105 expected after adaptation
ASR WER	14.2%	≈11–13% target
Retraining frequency	Monthly baseline	Monthly LM + weekly gazetteer
Manual annotation	3,000 initial tokens	+500–1,000 targeted code-mixed/name tokens per refresh

These target values should be verified on the real changed-condition test set. They are engineering targets rather than claims that can be guaranteed without the new corpus.

12D. Why modification is preferred

A complete replacement would discard the strong evidence that the deleted-interpolation model already captures local transport syntax effectively. The changed requirement mainly attacks vocabulary coverage and proper-name handling, so the most economical response is to add open-vocabulary and class-based mechanisms around the existing core.

The same reasoning applies to tagging. The Brill tagger has the best baseline overall accuracy, but its rules cannot invent lexical evidence for a previously unseen station. A gazetteer and character/suffix component directly address this failure mode. The design therefore separates stable syntactic knowledge from rapidly changing entity knowledge.

12D.1 Remaining limitations

- An n-gram model cannot represent arbitrarily long dependencies, discourse context or semantic intent.
- It may assign high probability to locally plausible but semantically incorrect destination sequences.
- Character models help with spelling variation but cannot guarantee that a phonetic form maps to the correct real-world entity.
- Code-mixed syntax may differ from English syntax, so a single English transition model can remain inadequate.
- ASR errors may be acoustic rather than linguistic; language-model rescoring cannot recover information that the recogniser has completely deleted.
- Proper-name gazetteers require continuous maintenance and may themselves become stale.

Conclusion

The transport enquiry problem demonstrates why language modelling and POS tagging must be evaluated as engineering systems rather than as isolated algorithms. Counting decisions determine vocabulary size; vocabulary size determines sparsity; sparsity motivates smoothing; smoothing and backoff determine whether unseen linguistic events can be scored; and the resulting language model directly affects ASR hypothesis selection.

For POS tagging, a domain-specific tagset provides a useful balance between linguistic detail and annotation cost. The rule-based tagger is transparent and fast, the HMM provides principled sequence decoding, and Brill transformation-based tagging gives the highest baseline accuracy while retaining readable rules. The final deployment recommendation is based on held-out measurements rather than accuracy alone.

The Day-4 challenge changes the problem substantially because new station names and code-mixed speech attack the assumptions of a fixed English vocabulary. The recommended response is modification: retain the successful deleted-interpolation core while adding a gazetteer, character-level fallback, class-based modelling and code-mixed tagging support.